Deep Dive in to

- List
- Dictionary
- Set
- Tuples

List

The list is Dynamic in size, meaning they can grow or shrink as needed. You can add or remove elements without having to specify the size beforehand. contain elements of different data types within the same list.

- Mutable
- Ordered, support indexing
- Hold Duplicate
- Hold different data types (int, str, float,bool)
- List Comprehension

Creation

Simple Method `abc = ["apple", "banana", "cherry", "orange"]`

by USING Constructor `abc = list(("apple", "banana", "cherry", "orange"))`

Perform some Operatoins/action on list

1. Access / Slicing Items
2. Loop Through
3. Length
4. Check if Item Exists
5. Changing the item
6. Adding an Item
7. Remove
8. Clear
9. Delete
10. Copy
11. Join
12. Number of elements in dataset
13. Index of the element with the specified value
14. Reverses the order
15. Sorts the list

List comprehension

List comprehension is a concise way **to create lists in Python**. It allows you to generate lists based on existing iterables like lists, strings, or other iterable objects. The syntax for list comprehension is compact and readable, making it a preferred choice over traditional loops in many cases.

```
[exp     for x in iterable ]
[x**2    for x in range(10)]

[exp     for x in iterable        if condition   ]
[x**2    for x in range(10)       if x % 2==0    ]

# Combine Multiple IF Conditions
Use of OR
[exp      for x in iterable    if cond1      or     cond2 ...   ]
[x        for x in range(10)   if x<2        or     x > 5       ]
Use of and
[exp      for x in iterable    if cond1      and    cond2 ...   ]
[x        for x in range(10)   if x%2 == 0   and    x > 5       ]

[exp      for x in iterable    if cond1             cond2 ...   ]
[x        for x in range(10)   if x%2 == 0          x > 5       ]


[exp1  if cond1   else  exp2 if cond2    else exp3      for x in iterable ]
[x*2   if x<5     else  x/2  if x<10     else x+2       for x in range(15)]
```

Tuples

Tuples in Python are ordered collections of elements, similar to lists, but with the key difference that tuples are immutable. This means that once a tuple is created, its elements cannot be changed, added, or removed. Tuples are defined by enclosing comma-separated elements within parentheses ().

- Not Mutable
- Hold Duplicate
- Ordered and support indexing
- Hold different data types(int, str, float,bool)

Creation

Simple Method `abc = ("apple", "banana", "cherry", "orange“)`

by USING Constructor `abc = tuple(("apple", "banana", "cherry", "orange"))`

Set

In Python, a set is an unordered collection of unique elements. It is defined using curly braces {}, with elements separated by commas. Sets are mutable, meaning you can add or remove elements after creation

- Mutable
- Do not hold Duplicate Values(Only unique)
- unordered and Do Not support indexing
- Hold different data types(int, str, float,bool)

Creation

Simple Method `abc = {"apple", "banana", "cherry", "orange“}`

by USING Constructor `abc = set(("apple", "banana", "cherry", "orange"))`

Dictionaries :

Example:

```
thisdict = {  "brand"   : "Ford",
              "model"   : "Mustang",
              "year"    :  1964 }
```

Dictionaries have **key-value pairs**, enabling **efficient lookup** and **retrieval** of values based on keys. establish them as fundamental tools in Python programming. This flexibility, combined with fast lookup times, makes dictionaries ideal for applications where **rapid access** to data is crucial.

- **Mutable in nature**
- **Unordered** collections, meaning they don't maintain a specific order of elements.
- **keys required to be unique**, and can hold **different datatypes**
- **Values can hold duplicate and accommodating various data types.**
- **Dynamic in Size**
- **dictionary comprehension,**

Creation

Simple dictionary

```
thisdict = {  "brand" : "Ford",
              "model" : "Mustang",
              "year"    :  1964 }

# Nested dictionary
DictFirst = { 'Dict1': {'name': 'Ali', 'age': 19},
              'Dict2': {'name': 'Bob', 'age': 25}}
```

by USING Constructor

```
thisdict = dict(brand="Ford", model="Mustang", year=1964)
```

Perform some action on Dictionary

1. Loop Through Dictionary
2. Length
3. Check if Item Exists
4. Changing the item
5. Adding an Item
6. Remove
7. Clear
8. Delete
9. Copy
10. Join

Dictionary comprehension

Dictionary comprehension is similar to list comprehension, but instead of creating lists, it creates dictionaries. It provides a **concise and efficient** way to generate dictionaries using an iterable and an expression to define key-value pairs. The syntax for dictionary comprehension is:

```
{ key_expression : value_expression for x in iterable     if condition }
```

```
{ x                : x**2             for x in Range(7)    if x % 2 == 0}
```

Example:

thisdict = {2: 4, 4: 16, 6: 36}